

Data Science — Lecture Notes

Compiled notes: Keyboard shortcuts, Data Science, Python, Statistics, SQL, Data Engineering

Keyboard Shortcuts

- Windows + X = All system details
- Alt + Tab = Switch to another window (Shift + Alt + Tab moves forward and backward)
- Windows + R = Run dialogue (command)
- Windows + H = Voice to text (in Microsoft Word)
- Windows + Shift + S = Snipping tool (edit screenshot while taking it)
- Windows + . = Emoji (can use in Word, Notion, Notepad)
- Ctrl + Shift + Esc = Task Manager
- Alt + Ctrl + Delete = when the system is hanging
- Alt + F4 = minimize; Alt + Fn + F4 = close all
- If a screen recording is less than 30 seconds, it can be made into a GIF
- Windows + Alt + R = Instant record
- Windows + G = Capture details + record screen option
- Ctrl + Shift + Arrow = select sentences (Shift + Arrow also works)
- Ctrl + Down Arrow = select the nearest sentence
- docs.new (search in Chrome) = new Google Document
- Ctrl + Shift + T = Redo/reopen an accidentally closed tab

Google Docs — Useful Tips

- Share criteria: can restrict others to view-only or allow editing
- Clock icon: check the change history (view changes)
- Multiple people can access; if a request is sent, it comes through via email
- @today — inserts today's date
- @yesterday — inserts yesterday's date
- File → New → Template
- Insert → Symbol (e.g. rupee symbol)
- Ctrl + Enter — next page
- File → Page setup — edit page as portrait, page size; make titles headings (Alt+Ctrl+1), make sub-headings
- To paste a paragraph or screenshot in the same format as the doc — Ctrl + Shift + V
- Ctrl + K — copy a link (site or picture, anything)
- Ctrl + / — shortcuts
- Ctrl + Alt + F — add a footnote
- Ctrl + F — Find any particular word
- Ctrl + H — Find and Replace any word

- Tab — move the cursor instead of using the mouse (works on any website)
- Windows → Accessibility → Virtual keyboard
- A Word file can be converted into Google Docs by uploading it to Drive

Other Notes

- NotebookLM (app) — if a PDF or file is large, it can summarize it; specific details can be asked
 - If you need any PDF file, use: filetype:pdf in search
-

Data Science

Dr. Iqnez Semmelweiz — 19 May 2026

Data Science is used in Travel, Healthcare, Sales, Credit & Media, Marketing, Social Media, and Automation.

- Automation — self-driving cars, pilotless aircrafts, drones
- Credit & Insurance — claims prediction, fraud & risk detection
- Marketing — upselling, cross-selling, churn (churn = a customer leaving or stopping a subscription)
- Data Science includes AI, ML, Deep Learning, Maths, Statistics, Visualization, and EDA

Why ML for the Future

- 460+ exabytes of data are generated every day, based on 2025 figures
- 1 byte = 8 bit
- 1000 such letters form 1 kilobyte

Machine Learning

1. Supervised Learning

Label the image, then train. Includes classification and regression.

- Classification — predicting a category or class. E.g. tiger vs. cat identification, email spam detection.
- Regression — predicting continuous numerical values. E.g. house price = ₹400000, salary = ₹20000, temperature = 45°C.

2. Unsupervised Learning

No labels — the machine figures out factors on its own. Includes clustering and association.

- Clustering — grouping similar data
- Association — finding relationships or rules between items

There is a site called "Teachable Machine" used to train models.

Machine Learning Applications

- Virtual Assistant
- eCommerce
- Financial Services
- Social Media

- Healthcare

Python

Python is a high-level, object-oriented, interpreted language with dynamic semantics, and is open source.

- NumPy — a Python library for multi-dimensional array processing / raw data handling (calculations, mathematical operations)
- Pandas — built on top of NumPy; provides multi-dimensional data structures for data analysis, cleaning, and manipulation
- scikit-learn — a machine learning library in Python (clustering, regression, etc.)

AI

- Turing Test
- Data Mining — collecting relevant data
- Data Cleaning — handling missing data
- Data Exploration (EDA) — forming a hypothesis about a defined problem (observing data and making possible assumptions or predictions)
- Feature Engineering — e.g. on weekends a lot of people purchase a particular thing; found through feature engineering (changing or creating data columns in a better way)
- Predictive Modelling — training a model using old data to predict future results
- Data Visualization

Lecture Notes — 22 May 2026

To Check if Python is Installed

```
Select terminal -> python --version OR py -V
```

If a version number is returned, Python is installed.

Naming Conventions

- Python style — snake_case
- JavaScript — camelCase

Virtual Environment

venv — built into Python.

Code Demo — Google Colab

Use "google colab.com" (search in Chrome) for code demos. Inserting a link is okay.

If you link an image, copy the address; the image can be shown on the right by adding "!" before it:

```
![Image] then paste the copied image address/link – the image will show on the right side
```

In Google Colab, extra data can be accessed through Drive: Files → Mount Drive → get data.

Colab / Python Basics

- In Colab, to comment out code, add a # symbol before the text
- For multiple output variables, use a bracket with commas — (4, 5, 6) or ("red", "yellow", "blue")
- Reuse / store → variables — a reserved memory location

```
age = 12
print(age)
```

Output = 12

Variable Rules

- Variables must start with a letter or underscore (not 0–9)
- Variables can't be a keyword like print
- Case sensitive

Operators

- Modulus % — finds the remainder
- Exponent ** — finds the power
- Floor division // — divides numbers and also removes the decimal part. E.g. 3.444 → 3
- == Equal to
- != Not equal to

(Relational operators)

Sets & Lists

In a set, we can't add duplicate elements. Sets support slicing conceptually only through lists; a set is unordered while a list is ordered and mutable.

List functions include a reverse function.

Loops

- Iteration method — used to avoid repeated print statements
- Definite loop — for loop: one condition, one increment/decrement
- Indefinite loop — while loop: initialization, condition (relational operation), increment or decrement

More than one condition can be combined using logical operators.

User Defined Functions

- Built-in functions

Control Statements (3 main types)

- Break
- Continue
- Pass

Function call example: mul() — the contents inside the parentheses are called arguments.

- Actual parameter

- Formal parameter

OOPs (Object-Oriented Programming)

- Map function, Object and class
 - Inside a class, a written function is called a method; variables are called attributes
 - Inheritance — multiple objects are created; parent class inherits to child class
 - Multilevel inheritance
 - Class access — the key is called objects
-

Version Control — Git & GitHub

02 June 2026

Version control repository — Local (PC/Laptop). Remote control repository — Remote (Server).

```
git status      -> shows the current state of your repository
git init        -> Git Initialize; creates a new Git repository in the current folder
git add
git commit -m "added file"
git add          (second line add)
git commit -m "added line1"
git log
git checkout
git switch
git branch
git checkout master
git branch
git branch dev
git checkout dev
git clone URL    -> gets code from a GitHub repository
git push
```

- Fork
 - Pull request
-

Statistics

08 June 2026

- Mean — Average
- Median — Middle value
- Mode — Most repeated value

Mean, Median, and Mode together are called measures of central tendency.

Standard Deviation

Standard Deviation (SD) measures how spread out the data is from the mean (average).

Example — Class A

Marks: 48, 49, 50, 51, 52. Mean = 50.

Distance from mean: 48 → 2 below; 49 → 1 below; 50 → exactly mean; 51 → 1 above; 52 → 2 above. All values are close to the mean.

Example — Class B

Marks: 20, 35, 50, 65, 80. Mean = 50.

Distance from mean: 20 → 30 below; 35 → 15 below; 50 → exactly mean; 65 → 15 above; 80 → 30 above. Values are much farther from the mean.

Worked Example: 2, 4, 6, 8, 10

Value	Difference from Mean	Difference Squared
2	-4	16
4	-2	4
6	0	0
8	2	4
10	4	16

Step 4 — Sum of squares = 40

Step 5 — Divide by number of values:

Variance = $40 / 5 = 8$

SD = $\sqrt{\text{Variance}} = \sqrt{8} = 2.83$

Variance vs. Standard Deviation

- Variance = average of squared deviations from the mean.
- Standard Deviation = square root of the variance.

Types of Statistics

- Descriptive — measures such as mean, median, mode, range, variance, standard deviation, and correlation
- Inferential — mostly used in data science

Pearson's Correlation Coefficient

Pearson's correlation coefficient (often denoted r) is a statistical measure that quantifies the strength and direction of the linear relationship between two continuous variables.

$$r = [n \cdot \sum xy - (\sum x)(\sum y)] / \sqrt{ [(n \cdot \sum x^2 - (\sum x)^2)(n \cdot \sum y^2 - (\sum y)^2)]}$$

Correlation strength ranges from Strong Positive, Medium Positive, Weak/No Correlation, Medium Negative, to Strong Negative.

Hypothesis Testing

Placebo

A placebo is any treatment that has no active properties, such as a sugar pill — a dummy treatment.

A placebo is a treatment with no active therapeutic ingredient, given in a way that makes a person believe they are receiving a real treatment.

| Source: betterhealth.vic.gov.au — Placebo effect

Hypothesis

Hypothesis — an assumption that is neither proven nor disproven. It is made at the very beginning, and the goal is to reject it or fail to reject it.

- H_0 = Null Hypothesis
- H_1 / H_a = Alternative Hypothesis

Null Hypothesis (H_0)

The starting assumption in hypothesis testing. States that there is no effect, no difference, or no relationship between variables.

- Assumes the current situation remains unchanged
- States no change or no impact
- Directly tested using sample data
- Written using equality symbols ($=$, $\geq$, $\leq$)

| Source: [geeksforgeeks.org](https://www.geeksforgeeks.org) — Null vs Alternate Hypothesis

Alternative Hypothesis (H_1 / H_a)

Challenges the null hypothesis. Suggests that an effect, difference, or relationship does exist.

- Represents the research or study claim
- Indicates that a change or effect is present
- Accepted only when the null hypothesis (H_0) is rejected
- Written using inequality symbols ($\neq$, $>$, $<$)

| Source: [geeksforgeeks.org](https://www.geeksforgeeks.org) — Null vs Alternate Hypothesis

Hypothesis Testing: either reject the null hypothesis or fail to reject it.

- Reject the null hypothesis → accepting the alternative hypothesis
- Fail to reject the null hypothesis → accepting the null hypothesis

Example: A doctor would not prescribe diabetic medicine until the blood report confirms diabetes.

Significance Level (α)

In statistics, the significance level (denoted α , alpha) is the probability of rejecting the null hypothesis when it is actually true (a Type I error).

- $\alpha = 0.05$ (5%) — most commonly used
- $\alpha = 0.01$ (1%) — stricter evidence required
- $\alpha = 0.10$ (10%) — less strict, sometimes used in exploratory studies

Decision rule:

- If $p\text{-value} \leq \alpha$, reject the null hypothesis.

- If $p\text{-value} > \alpha$, fail to reject the null hypothesis.

Example

Suppose $\alpha = 0.05$ and a hypothesis test gives a $p\text{-value}$ of 0.03. Since $0.03 < 0.05$, you reject the null hypothesis and conclude the result is statistically significant at the 5% level.

In simple terms, a significance level of 0.05 means you are willing to accept a 5% chance of falsely concluding an effect exists when it actually does not.

Significance level (α) is the maximum probability of rejecting a true null hypothesis (Type I error).

- $p\text{-value}$ = the actual probability that the result occurred by random chance.
- Significance level (α) = the cutoff value we choose (usually 0.05).

Decision: $p\text{-value} \leq \alpha \rightarrow \text{Reject } H_0$; $p\text{-value} > \alpha \rightarrow \text{Do not reject } H_0$.

So, "possibility of randomness" is closer to the $p\text{-value}$, while the significance level is the allowed limit for that possibility.

Example — Battery Life Claim

A company claims that the average battery life of its product is 10 hours.

- Null Hypothesis (H_0): average battery life = 10 hours
- Alternative Hypothesis (H_1): average battery life $\neq$ 10 hours

Chosen significance level: 5% — "I am willing to accept a 5% risk of wrongly rejecting the null hypothesis."

After collecting data: $p\text{-value} = 0.03$. Since $0.03 < 0.05$, reject H_0 .

Interpretation: Because the $p\text{-value}$ is smaller than the significance level, the observed result is considered too unlikely to be explained by random chance alone.

Easy Way to Remember

Imagine a teacher says: "If the chance that this happened randomly is less than 5%, I'll consider it significant."

- 5% = Significance level
- Actual chance from the data = $p\text{-value}$

If the actual chance ($p\text{-value}$) is below 5%, reject H_0 . If it's above 5%, don't reject H_0 .

P-value Formula

$P\text{-value}$ is short for probability value. It defines the probability of getting a result equal to or more extreme than the actual observations. The $P\text{-value}$ represents the probability of the given event occurring. It is used as an alternative to the rejection point to determine the least significance level at which the null hypothesis would be rejected. The smaller the $p\text{-value}$, the stronger the evidence in favor of the alternative hypothesis.

$P\text{-value}$ is an important statistical measure that helps determine whether the hypothesis is correct. It always lies between 0 and 1. The level of significance (α) is a predefined threshold set by the researcher, generally fixed at 0.05.

Source: cuemath.com — *P-value Formula*

Confidence Level

Confidence Level = the percentage that indicates how confident we are that the sample result represents the true population parameter.

- 95% confidence level = standard choice
- 99% confidence level = more strict
- 90% confidence level = less strict

Example: Suppose you want to know the average height of 10,000 students. You can't measure everyone, so you take a sample of 100 students. After calculation, you estimate: average height $\approx$ 165 cm.

If you say 95% confidence level, it means: "I am 95% confident that my estimation method is correct." The remaining 5% is the chance that your sample gave a misleading result.

- Confidence Level = 95% $\rightarrow$ Confidence in the result
- Significance Level = 5% $\rightarrow$ Risk of being wrong

Z-score

A Z-score (Standard Score) tells you how many standard deviations a data point is away from the mean.

$$z = (x - \mu) / \sigma$$

- z = Z-score
- x = data value
- μ (mu) = mean
- σ (sigma) = standard deviation

Z-Score Formula is not a hypothesis test — use it when finding how far a single value is from the mean.

Hypothesis Testing — Types

Z Test

One-Sample Z-Test for Mean

Use when: population SD (σ) is known — testing an average.

Z-Test Formulas

Test Type	Formula	When to Use
1. One-Sample Z-Test for Mean	$z = (\bar{x} - \mu_0) / (\sigma/\sqrt{n})$	Population SD (σ) known; testing a population mean (average)
2. One-Sample Z-Test for Proportion	$z = (\hat{p} - p_0) / \sqrt{p_0(1-p_0)/n}$	Testing a population proportion (percentage); large sample ($np_0 \geq 5$, $n(1-p_0) \geq 5$)
3. Two-Sample Z-Test for Means	$z = (\bar{x}_1 - \bar{x}_2) / \sqrt{(\sigma_1^2/n_1 + \sigma_2^2/n_2)}$	Comparing means of two independent populations; population SDs known
4. Two-Proportion Z-Test	$z = (\hat{p}_1 - \hat{p}_2) / \sqrt{\hat{p}(1-\hat{p})(1/n_1 + 1/n_2)}$, where $\hat{p} = (x_1 + x_2) / (n_1 + n_2)$	Comparing proportions of two independent populations; large samples
5. Z-Score Formula (not a hypothesis test)	$z = (x - \mu) / \sigma$	Finding how far a single value is from the mean; standardization; finding probabilities in a normal distribution

Common significance levels (α): $0.10 \rightarrow z = 1.645$ | $0.05 \rightarrow z = 1.960$ | $0.01 \rightarrow z = 2.576$ | $0.001 \rightarrow z = 3.291$

Population Mean (μ)

This is the already-given claim / reference value in the question.

- "The company claims..."
- "The average is..."
- "The population mean is..."

Example: "The average salary is ₹30,000." $\rightarrow \mu = 30000$ (Population Mean)

Sample Mean ($\bar{x}$)

This is the average calculated from the sample taken in the question.

- "A sample of 25 employees..."
- "The sample mean was..."
- "The average of the sampled data..."

Example: "A sample of 25 employees had an average salary of ₹32,000." $\rightarrow \bar{x} = 32000$ (Sample Mean)

Easy Memory Trick

- μ (mu) = Mentioned / Market claim — already known value, population value
- $\bar{x}$ (x-bar) = eXperiment value — the value obtained from the sample, data we collected

T Test

$$t = (\bar{x} - \mu) / (s/\sqrt{n})$$

T-Test Steps (One-Sample t-Test for Mean)

1. State the Hypotheses — $H_0: \mu = \mu_0$; H_1 depends on the claim: $\mu > \mu_0$ (right-tailed), $\mu < \mu_0$ (left-tailed), or $\mu \neq \mu_0$ (two-tailed)
2. Choose the Significance Level (α) — commonly 0.05 or 0.01
3. Check Conditions — random sample; population approximately normal OR $n \geq 30$ (Central Limit Theorem); independence of observations
4. Compute the Test Statistic — $t = (\bar{x} - \mu_0) / (s/\sqrt{n})$, where $\bar{x}$ = sample mean, μ_0 = claimed population mean, s = sample standard deviation, n = sample size
5. Determine the Degrees of Freedom (df) — $df = n - 1$
6. Find the P-value (using t-table or software) — right-tailed: $p = P(T > t_{\text{calc}})$; left-tailed: $p = P(T < t_{\text{calc}})$; two-tailed: $p = 2 \cdot P(T > |t_{\text{calc}}|)$
7. Make the Decision — if $p \leq \alpha$, reject H_0 ; if $p > \alpha$, fail to reject H_0
8. Draw the Conclusion — state it in context: if H_0 is rejected, there is sufficient evidence to support the claim; if not rejected, there is insufficient evidence.

Rejection Regions (by hypothesis)

- Right-tailed test ($H_1: \mu > \mu_0$) — reject H_0 if $t_{\text{calc}} > t_{\alpha, df}$
- Left-tailed test ($H_1: \mu < \mu_0$) — reject H_0 if $t_{\text{calc}} < -t_{\alpha, df}$
- Two-tailed test ($H_1: \mu \neq \mu_0$) — reject H_0 if $|t_{\text{calc}}| > t_{\alpha/2, df}$

Note: For large samples ($n \geq 30$), the t-test is approximately the same as the z-test.

Matrix — Collection of Vectors

Matrices are used in data science to represent datasets and perform efficient mathematical computations in machine learning, image processing, and dimensionality-reduction techniques such as PCA.

A matrix is a rectangular arrangement of numbers in rows and columns. In data science, matrices are used to store and process data efficiently.

Main Uses

1. Representing Data

- Rows represent observations (records).
- Columns represent features (variables).
- Example: a dataset containing age, salary, and experience can be stored as a matrix.

2. Machine Learning

- Algorithms such as Linear Regression, Logistic Regression, and Neural Networks perform calculations using matrices.
- Matrix operations make computations faster and more efficient.

3. Image Processing

- Images are stored as matrices of pixel values.
- Each pixel's intensity or color value is represented in a matrix.

4. Dimensionality Reduction

- Techniques like Principal Component Analysis (PCA) use matrices, eigenvalues, and eigenvectors to reduce the number of features while preserving important information.

5. Data Transformations

- Scaling, rotation, and other mathematical transformations are performed using matrices.

"Principal Component Analysis (PCA) is a dimensionality reduction technique that converts a large set of correlated variables into a smaller set of uncorrelated variables called principal components while preserving most of the variance present in the original data."

Why Data Science Uses Matrices

- PCA (Principal Component Analysis) uses eigenvalues and eigenvectors to reduce dimensions.
- Image compression.
- Face recognition.
- Recommendation systems.
- Feature extraction.

In PCA:

- Eigenvectors → directions of maximum variance.
- Eigenvalues → how much variance exists in those directions.

An eigenvector is a vector whose direction does not change after a matrix transformation. An eigenvalue is the scalar that tells how much the eigenvector is stretched or compressed. They are commonly used in PCA and machine learning.

Differentiation & Optimization

Differentiation

Differentiation is the process of finding the rate at which a function changes with respect to a variable.

It is a calculus technique used to measure the rate of change of a function. In data science, it is mainly used to calculate gradients and optimize machine learning models by minimizing errors.

Why Data Scientists Learn Differentiation

- To understand Gradient Descent
- To minimize loss/error functions
- To train Machine Learning and Neural Network models
- To understand optimization techniques

Optimization

Optimization means finding the maximum or minimum value of a function. In Data Science, we usually want to minimize the error (loss) of a machine learning model. Graphically, optimization means finding the lowest point (minimum) or highest point (maximum) on a curve.

Slopes at Different Points

- Before the maximum point → positive slope (+); graph is increasing.
- At the maximum point → slope = 0; horizontal tangent.
- Between maximum and minimum → negative slope (-); graph is decreasing.
- At the minimum point → slope = 0; horizontal tangent.
- After the minimum point → positive slope (+); graph is increasing again.

Optimization Rule

9. Find the derivative.
10. Set the derivative equal to zero.
11. Solve for critical points.
12. Check whether each point is a maximum or minimum.

Probability

Probability is the measure of how likely an event is to occur, ranging from 0 (impossible) to 1 (certain).

$P(E) = \text{Number of favorable outcomes} / \text{Total number of possible outcomes}$

Examples

- Tossing a fair coin: Probability of Heads = $1/2 = 0.5$
- Drawing an Ace from a standard 52-card deck: Probability = $4/52 = 1/13$

Law of Large Numbers

As the number of trials or observations increases, the sample average gets closer and closer to the true population mean (expected value).

Example — tossing a fair coin:

- 10 tosses → Heads might appear 7 times → Probability = $7/10 = 0.7$
- 100 tosses → Heads might appear 52 times → Probability = $52/100 = 0.52$
- 1000 tosses → Heads might appear 498 times → Probability = $498/1000 = 0.498$

As the number of tosses increases, the experimental probability approaches the true probability of 0.5.

Mutually Exclusive Events

Two events are called mutually exclusive if they cannot occur at the same time.

Example — Rolling a Die

Let A = getting an even number = {2, 4, 6}; B = getting an odd number = {1, 3, 5}. A and B cannot happen together in a single roll, so they are mutually exclusive.

- Mutually Exclusive: cannot happen together.
- Independent: occurrence of one does not affect the other.

Conditional Probability

Conditional probability is the probability of an event occurring given that another event has already occurred:

$$P(A|B) = P(A \cap B) / P(B)$$

Source: LinkedIn Learning — Machine Learning Foundations: Probability, glossary of terms

Dependent Events

Dependent events are events where the occurrence of one affects the probability of the other.

Example: In image classification, if an image contains "wheels," the probability it contains a "car" is higher — the presence of wheels increases the chance of a car being present.

Source: LinkedIn Learning — Machine Learning Foundations: Probability, glossary of terms

Q&A

09 June 2026

Q4. A company claims that the average website load time is 2 seconds. A sample of 50 users shows an average load time of 2.3 seconds. The population standard deviation is 0.5 seconds. Test at $\alpha = 0.05$ whether the claim holds.

Given

- $\mu_0 = 2$ seconds

- $\bar{x} = 2.3$ seconds
- $\sigma = 0.5$ seconds
- $n = 50$
- $\alpha = 0.05$

Step 1: Hypotheses

- $H_0: \mu = 2$
- $H_1: \mu \neq 2$ (two-tailed test)

Step 2: Calculate Z-value

$$z = (\bar{x} - \mu_0) / (\sigma/\sqrt{n}) = (2.3 - 2) / (0.5/\sqrt{50}) \approx 4.24$$

Step 3: Critical Value

For a two-tailed test at $\alpha = 0.05$, the critical z-value is ± 1.96 .

Step 4: Decision

Since the calculated z (≈ 4.24) exceeds the critical value (1.96), reject H_0 .

Conclusion

At the 5% significance level, there is sufficient evidence to conclude that the average website load time has changed from 2 seconds.

Answer: Reject H_0 . The average website load time is significantly different from 2 seconds.

NumPy

15 June 2026

NumPy (Numerical Python) is a Python library used for numerical and mathematical computations. It provides support for large, multi-dimensional arrays and matrices along with various mathematical functions.

Various other libraries like Pandas, Matplotlib, and scikit-learn are built on top of this library.

```
import numpy as np
```

In Python, "as" is used to give an alias (short name) to a module, package, or object when importing.

Installation command:

```
pip install numpy
```

Arrays

- One-dimensional — vector
- Two-dimensional — matrix

NumPy arrays are called ndarray (N-dimensional arrays) and store elements of the same type and size.

NumPy is Regularly Applied For

- Data manipulation and analysis: for data cleaning, transformation, and aggregation

- Scientific computing: handling advanced mathematical operations such as matrix multiplication, eigenvalues
- Machine learning
- Signal and image processing

Array Creation

```
import numpy as np
np.array([1, 2, 3])
np.zeros((2, 3))
np.ones((3, 3))
np.empty((2, 2))
np.arange(0, 10, 2)
np.linspace(0, 1, 5)
np.eye(3)
np.random.rand(2, 3)
np.random.randint(1, 10, (2, 2))
```

np.empty() is used to allocate memory for large arrays when the values will be assigned later. It improves performance because NumPy does not initialize the array elements.

Array Inspection

```
a.shape    # Dimensions
a.ndim     # Number of dimensions
a.size     # Total elements
a.dtype    # Data type
```

Array Reshaping

```
a.reshape(2, 3)
a.flatten()
a.ravel()
a.T    # Transpose
```

Mathematical Operations

```
np.add(a, b)
np.subtract(a, b)
np.multiply(a, b)
np.divide(a, b)
np.power(a, 2)
np.sqrt(a)
np.exp(a)
np.log(a)
```

Statistical Functions

```
np.mean(a)
np.median(a)
np.std(a)
np.var(a)
np.min(a)
np.max(a)
np.sum(a)
```

```
np.cumsum(a)
```

Aggregation Along Axis

```
np.sum(a, axis=0)    # Column-wise  
np.sum(a, axis=1)    # Row-wise  
np.mean(a, axis=0)  
np.max(a, axis=1)
```

Searching & Sorting

```
np.sort(a)  
np.argsort(a)  
np.argmax(a)  
np.argmin(a)  
np.where(a > 5)
```

Joining Arrays

```
np.concatenate((a, b))  
np.vstack((a, b))  
np.hstack((a, b))
```

Splitting Arrays

```
np.split(a, 2)  
np.vsplit(a, 2)  
np.hsplit(a, 2)
```

Linear Algebra

```
np.dot(a, b)  
np.matmul(a, b)  
np.linalg.inv(a)  
np.linalg.det(a)  
np.linalg.eig(a)
```

Pandas

15 June 2026

Pandas is an open source library built on top of NumPy. It is a package that offers various data structures and operations for manipulating tabular data and time series (e.g. temperature, stock market) and many other data types.

It is popular for importing, analyzing, cleaning, exploring, and manipulating data. Pandas provides Series (1D) and DataFrame (2D) structures.

```
import pandas as pd
```

Advantages

- Data from different sources can be loaded
- Easy handling of missing data (represented as NaN)

- Size mutability: columns can be inserted/deleted from a DataFrame
- Dataset merging and joining
- Allows analyzing big data and drawing conclusions based on statistical theories efficiently
- Pandas can clean messy datasets, making them readable and relevant
- Relevant data is very important in data science

Series vs DataFrame

Series	DataFrame
One-dimensional (1D)	Two-dimensional (2D)
Contains a single column of data	Contains multiple columns of data
Similar to a Python list or NumPy array with labels	Similar to an Excel sheet or SQL table
Has an index and values	Has row index and column names
Created using <code>pd.Series()</code>	Created using <code>pd.DataFrame()</code>

1. Series

A Series is a one-dimensional labeled array.

```
import pandas as pd
s = pd.Series([10, 20, 30, 40])
print(s)
```

Output:

```
0    10
1    20
2    30
3    40
```

dtype: int64

You can also specify custom indexes:

```
s = pd.Series([90, 85, 95], index=["John", "Alice", "Bob"])
print(s)
```

Output:

```
John    90
Alice   85
Bob     95
dtype: int64
```

2. DataFrame

A DataFrame is a two-dimensional table consisting of rows and columns.

```
import pandas as pd
data = {
    "Name": ["John", "Alice", "Bob"],
    "Age": [22, 21, 23],
    "Marks": [90, 85, 95]
}
df = pd.DataFrame(data)
print(df)
```

Output:

```
   Name  Age  Marks
```

```
0   John   22   90
1  Alice   21   85
2   Bob   23   95
```

Relationship between Series and DataFrame: a DataFrame is a collection of Series.

Commonly Used Pandas Functions

1. Reading Data

```
pd.read_csv()
pd.read_excel()
pd.read_json()
```

2. Viewing Data

```
df.head()      # First 5 rows
df.tail()      # Last 5 rows
df.info()      # DataFrame information
df.describe()  # Statistical summary
df.shape       # Rows and columns
df.columns     # Column names
```

3. Selecting Data

```
df['Name']      # Single column
df[['Name', 'Age']] # Multiple columns
df.loc[0]       # Row by label
df.iloc[0]      # Row by index
```

4. Filtering

```
df[df['Age'] > 25]
df[df['City'] == 'Delhi']
```

5. Sorting

```
df.sort_values('Age')
df.sort_values('Age', ascending=False)
```

6. Missing Values

```
df.isnull()
df.isnull().sum()
df.dropna()
df.fillna(0)
```

7. Aggregation

```
df['Salary'].mean()
df['Salary'].sum()
df['Salary'].max()
df['Salary'].min()
df['Salary'].count()
```

8. Grouping

```
df.groupby('Department')['Salary'].mean()
df.groupby('City').count()
```

9. Unique Values

```
df['City'].unique()  
df['City'].nunique()
```

10. Column Operations

```
df['Bonus'] = df['Salary'] * 0.1  
df.rename(columns={'Name': 'Employee_Name'})  
df.drop('Bonus', axis=1)
```

11. Merging & Joining

```
pd.merge(df1, df2, on='ID')  
df1.join(df2)
```

12. Duplicate Handling

```
df.duplicated()  
df.drop_duplicates()
```

13. Value Counts

```
df['City'].value_counts()
```

14. Correlation

```
df.corr()
```

.map() Function

The .map() function is used to transform or replace values in a Series (single column).

query()

The query() method is used to filter rows based on a condition. It is an alternative to Boolean indexing and often makes conditions easier to read.

Example — salary greater than 65000:

Using Boolean indexing:

```
dd[dd["salary"] > 65000]
```

Using query():

```
dd.query("salary > 65000")
```

Database & SQL

19 June 2026

Database = a structured collection of data that is stored electronically and can be easily accessed, managed, updated, and retrieved.

Used For

- Improve business processes
- Keep track of customers

- Store personal data
- Keep a record of music or book collections
- Finances
- Healthcare
- Weather
- Government organizations
- Based on historical data — can manage, retrieve, etc.

Types of Databases

- Relational databases — structured database
- Object-oriented databases
- Distributed database
- Data warehouses
- NoSQL databases — unstructured database

SQL

- Structured Query Language
- Used to access and manipulate databases
- E.g. MySQL, Oracle

What Can SQL Do?

- Execute queries against a database
- Retrieve data from a database
- Insert records into a database
- Update records in a database
- Delete records from a database
- Create new databases
- Create new tables in a database

Important Terminology

- Database tables are referred to as relations
- The columns of a table are referred to as attributes or fields
- Rows are referred to as tuples or records
- The number of attributes in a relation is known as the degree of the relation
- The number of tuples in a relation is known as the cardinality
- Entity — an object that exists in the real world; a single table or particular entity

Types of Keys

- Primary key — can't allow null values; used for relations
- Foreign key
- Composite key

- Unique key — same as foreign, but can allow 1 null value

Data Types

- Int
- Varchar
- Char
- Boolean
- Float
- Date
- Datetime

Database Languages

- Data Definition Language (DDL) — used when creating or changing a data structure: Create, Alter, Drop
- Data Manipulation Language (DML) — used to store, modify, retrieve, delete, and update data: Select, Insert, Update, Delete
- Data Control Language (DCL) — used for access permission control: Grant, Revoke
- Transaction Control Language (TCL) — deals with transactions within a database: Commit, Rollback, Savepoint

SQL Commands

- SELECT
- UPDATE
- DELETE
- INSERT INTO — adds new records (rows) into a table
- CREATE DATABASE
- ALTER DATABASE — modifies a database
- CREATE TABLE
- ALTER TABLE — modifies a table
- DROP TABLE — deletes the whole table
- TRUNCATE TABLE — keeps the structure but deletes the contents of the table

Sample Workflow

First create a database → then a table → INSERT INTO() ... VALUES.

- SELECT * FROM table — show all contents in the table
- WHERE — condition
- DESC — get results in descending order
- LIMIT 2 — get 2 rows
- DISTINCT — get particular / unique values
- ORDER BY — used to sort data
- GROUP BY — used for grouping

ALTER vs UPDATE

- ALTER → changes the table structure (column add, rename, datatype change...)
- UPDATE → changes the data inside the table

```
ALTER TABLE Patients
RENAME COLUMN Age TO patient_age;

ALTER TABLE Patients
ADD Email VARCHAR(50);

ALTER TABLE Patients
DROP COLUMN Email;

ALTER TABLE Patients
MODIFY Contact_No BIGINT;

ALTER TABLE Patients
RENAME TO Patient_Details;  -- table rename

ALTER TABLE Patients
MODIFY patient_id INT AUTO_INCREMENT;
```

UPDATE Patterns

```
UPDATE ... WHERE ...
UPDATE ... IN (...)
UPDATE ... LIKE ...
UPDATE ... IS NULL
-- UPDATE multiple columns
```

IN Operator

IN is used to check multiple values in a single condition. Instead of writing:

```
SELECT * FROM Patients
WHERE patient_id = 113
OR patient_id = 123
OR patient_id = 145;
```

You can write:

```
SELECT * FROM Patients
WHERE patient_id IN (113, 123, 145);
```

Example: SELECT

```
SELECT * FROM Doctors
WHERE DoctorID IN (2345, 2346);
```

Output: shows doctors whose ID is either 2345 or 2346.

LIKE — Pattern Matching

% = any number of characters.

Starts with A:

```
SELECT * FROM Patients
```

```
WHERE patient_name LIKE 'A%';  
-- Matches: Arjun, Anu, Amrutha  
Ends with n:
```

```
SELECT * FROM Patients  
WHERE patient_name LIKE '%n';  
-- Matches: Arjun
```

Contains "ru":

```
SELECT * FROM Patients  
WHERE patient_name LIKE '%ru%';  
-- Matches: Arjun
```

ORDER BY

Used to sort the records in a table in ascending or descending order.

Sort by Age (Descending):

```
SELECT * FROM Patients  
ORDER BY patient_age DESC;  
-- Output: highest age to lowest age
```

Sort by Age (Ascending):

```
SELECT * FROM Patients  
ORDER BY patient_age ASC;  
-- Output: lowest age to highest age
```

GROUP BY

Used to group rows that have the same value in a column; usually works with aggregate functions like COUNT(), SUM(), AVG(), MAX(), MIN().

```
SELECT Doctor_name, COUNT(*)  
FROM Doctors  
GROUP BY Doctor_name;  
-- Output: Arun 1, Midhun 1, Sreejith 1  
SELECT Specialization, COUNT(*) AS total  
FROM Doctors  
GROUP BY Specialization;
```

Specialization	Total
Cardio	3
Ortho	2

- Specialization → shows the specialization name.
- COUNT(*) → counts the number of rows (doctors).
- AS total → gives the count column the name total.

| Source: [chatgpt.com conversation](#)

WHERE — Applying Conditions

Filters rows.

```
SELECT * FROM Patients  
WHERE Age > 25;
```

Comparison Operator: <> (Not Equal)

```
SELECT * FROM Patients
WHERE Gender <> 'Male';
```

BETWEEN — Range of Values

```
SELECT * FROM Patients
WHERE Age BETWEEN 20 AND 30;

SELECT * FROM Appointments
WHERE AppointmentDate
BETWEEN '2026-06-20' AND '2026-06-22';
```

DISTINCT — Remove Duplicates

```
SELECT DISTINCT Gender
FROM Patients;
```

Differences in Terms

- MAX(Specialization) → alphabetically largest text — here in the specialization column, whichever text is "largest" by length
- MAX(patient_age) → largest age value
- COUNT(*) → number of rows
- GROUP BY + COUNT(*) + ORDER BY DESC → most frequent value (very common in Data Science)

CASE WHEN — SQL's IF-ELSE Statement

Very common in Data Analytics and Data Science interviews.

```
SELECT column_name,
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE result3
END AS new_column
FROM table_name;
```

Example:

```
SELECT patient_name,
patient_age,
CASE
  WHEN patient_age < 18 THEN 'Child'
  WHEN patient_age BETWEEN 18 AND 59 THEN 'Adult'
  ELSE 'Senior Citizen'
END AS Age_Group
FROM patients;
```

HAVING

- WHERE → filters rows
- HAVING → filters groups (used after GROUP BY, to filter aggregated data)

WHERE	HAVING
Filters individual rows	Filters groups

WHERE	HAVING
Works before GROUP BY	Works after GROUP BY
Cannot use aggregate functions	Can use aggregate functions

```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name
HAVING COUNT(*) > 1;
```

Source: chatgpt.com conversation

Joins

A JOIN is used to combine rows from two or more tables based on a related column.

1. INNER JOIN

Returns only matching rows from both tables — fetches common information.

```
SELECT column1, column2, ...
FROM table1
INNER JOIN table2
ON table1.common_column = table2.common_column;
```

2. LEFT JOIN (LEFT OUTER JOIN)

Takes all information from the first table plus the common things from the right table. Returns all rows from the left table and matching rows from the right table.

```
SELECT column1, column2
FROM table1
LEFT JOIN table2
ON table1.id = table2.id
```

3. RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table and matching rows from the left table.

```
SELECT column1, column2
FROM table1
RIGHT JOIN table2
ON table1.id = table2.id
```

4. FULL JOIN (FULL OUTER JOIN)

Returns all rows from both tables, matched where possible.

```
SELECT column1, column2
FROM table1
OUTER JOIN table2
ON table1.id = table2.id
```

Window Functions in SQL

Window functions perform calculations across a set of rows related to the current row, without grouping the rows into a single result.

```
function_name() OVER(  
  [PARTITION BY column]  
  [ORDER BY column]  
)
```

RANK() vs COUNT() OVER()

RANK() is a window function that assigns a rank to each row based on the specified order. Rows with the same value receive the same rank, and the next rank is skipped.

COUNT() OVER() calculates the count as a window function and returns the count for each row without collapsing rows, unlike GROUP BY.

COUNT — with GROUP BY

```
SELECT Department,  
       COUNT(*) AS DeptCount  
FROM Members  
GROUP BY Department;
```

Department	DeptCount
CSE	2
ECE	2
EEE	1

COUNT() OVER() — without GROUP BY

```
SELECT student_name,  
       Department,  
       COUNT(*) OVER(PARTITION BY Department) AS DeptCount  
FROM Members;
```

student_name	Department	DeptCount
Lekshmi	CSE	2
Anu	CSE	2
Meenu	ECE	2
Anju	ECE	2
Sera	EEE	1

Result keeps all rows and shows the count beside each row. This is the main difference between GROUP BY and COUNT() OVER().

| Source: chatgpt.com conversation

1. ROW_NUMBER()

Gives a unique number to each row.

```
SELECT student_name,  
       ROW_NUMBER() OVER(ORDER BY student_name) AS RowNo  
FROM Members;
```

student_name	RowNo
Anju	1
Anu	2

student_name	RowNo
Lekshmi	3
Meenu	4

2. RANK()

Same values get the same rank; the next rank is skipped.

```
SELECT student_name,
       RANK() OVER(ORDER BY Marks DESC) AS RankNo
FROM Students;
```

Marks	Rank
100	1
95	2
95	2
90	4

3. DENSE_RANK()

Same values get the same rank, but no rank is skipped.

```
SELECT student_name,
       DENSE_RANK() OVER(ORDER BY Marks DESC) AS RankNo
FROM Students;
```

Marks	Dense Rank
100	1
95	2
95	2
90	3

4. COUNT() OVER()

Counts rows without grouping.

```
SELECT student_name,
       COUNT(*) OVER() AS TotalMembers
FROM Members;
```

5. SUM() OVER()

Running total.

```
SELECT SaleDate,
       Amount,
       SUM(Amount) OVER(ORDER BY SaleDate) AS RunningTotal
FROM Sales;
```

6. AVG() OVER()

Average value for each row.

```
SELECT student_name,
       Marks,
       AVG(Marks) OVER() AS AvgMarks
```

```
FROM Students;
```

7. MIN() OVER() and MAX() OVER()

```
SELECT student_name,  
       Marks,  
       MIN(Marks) OVER() AS MinMarks,  
       MAX(Marks) OVER() AS MaxMarks  
FROM Students;
```

8. LAG()

Gets the previous row's value.

```
SELECT SaleDate,  
       Amount,  
       LAG(Amount) OVER(ORDER BY SaleDate) AS PreviousSale  
FROM Sales;
```

9. LEAD()

Gets the next row's value.

```
SELECT SaleDate,  
       Amount,  
       LEAD(Amount) OVER(ORDER BY SaleDate) AS NextSale  
FROM Sales;
```

Reading Data from Various Sources

24 June 2026

Data Source	Function	Example
CSV file	pd.read_csv()	df = pd.read_csv("employees.csv")
Excel file	pd.read_excel()	df = pd.read_excel("employees.xlsx")
JSON file	pd.read_json()	df = pd.read_json("data.json")
SQL Database	pd.read_sql()	df = pd.read_sql("SELECT * FROM employee", connection)
HTML Table	pd.read_html()	tables = pd.read_html("https://example.com")
XML file	pd.read_xml()	df = pd.read_xml("data.xml")
Parquet file	pd.read_parquet()	df = pd.read_parquet("data.parquet")
Clipboard	pd.read_clipboard()	df = pd.read_clipboard()
Fixed-width text	pd.read_fwf()	df = pd.read_fwf("data.txt")

Source: chatgpt.com conversation

Data Engineering vs Data Science vs Machine Learning

24 June 2026

Data Engineering is the field of designing, building, and maintaining systems that collect, store, process, and deliver data for analytics, machine learning, and business applications.

	Data Engineering	Data Science	Machine Learning
Primary Focus	Build and maintain scalable data systems and infrastructure.	Analyze data and extract meaningful insights to solve business problems.	Build models that learn from data and make predictions or decisions.
Key Responsibilities	Design data pipelines (ETL/ELT); collect, ingest, store data; build data warehouses/lakes; ensure quality & scalability; optimize workflows	Clean and explore data; perform statistical analysis; create reports & dashboards; identify patterns & trends; communicate insights	Prepare data for modeling; build & train ML models; evaluate & tune models; deploy & monitor models; improve models over time
Typical Output	Data pipelines, data warehouse/lake, clean & reliable datasets	Reports, dashboards, insights & recommendations	Trained models, predictions, automated predictions
Common Tools	Python, SQL, Spark, Airflow, Kafka, dbt, AWS, Azure, GCP, Snowflake	Python, SQL, Pandas, Excel, Jupyter, Matplotlib, Seaborn, Tableau, Power BI, R	Python, Scikit-learn, TensorFlow, PyTorch, Keras, XGBoost, MLflow, Hugging Face
Requires Strong In	Databases, SQL, Python/Scala, system design, cloud, distributed systems	Statistics, data analysis, visualization, domain knowledge, communication	Algorithms, statistics, linear algebra, modeling, programming
Key Question Answered	How do we collect, store, and deliver data reliably and efficiently?	What happened? Why did it happen? What are the key trends & insights?	What will happen next? What should we do? Can we automate or predict it?
Who Uses Their Work	Data Analysts, Data Scientists, BI Tools, Business Applications	Business teams, analysts, management, product teams	Applications, end users, automated systems
In Simple Words	Builds the foundation (collects and prepares the data).	Turns data into insights (understands the data).	Uses insights to make predictions and automate decisions.

Pipeline — flow of data.

ETL and ELT are two methods for moving and preparing data. The main difference is when the data is transformed.

ETL (Extract → Transform → Load)



Example:

- Extract customer data from a website.
- Transform: remove duplicates, fix missing values, standardize dates.
- Load the cleaned data into a data warehouse.

ELT (Extract → Load → Transform)

```
Data Sources
|
v
Extract
|
v
Load (Data Warehouse / Data Lake)
|
v
Transform (SQL, Spark, dbt, etc.)
```

Example:

- Extract raw customer data.
- Load it directly into a cloud data warehouse.
- Transform it later using SQL or other processing tools.

ML Stack Overview

- Framework: TensorFlow, PyTorch, scikit-learn
- MLOps & Deployment: Docker, Kubernetes, CI/CD Pipelines
- Core Languages: Python (with strong software-engineering fundamentals), R, SQL, Julia, Scala, Java, C++

Data Science: Framework, Core Languages & Deployment

1. Data Science Framework

13. Define Problem — understand the business problem; define objectives & success metrics
14. Collect Data — identify data sources; gather relevant data
15. Explore & Understand Data — EDA (exploratory data analysis); data profiling & visualization
16. Prepare Data — clean data, handle missing values; feature engineering
17. Build Model — select algorithms; train models
18. Evaluate Model — validate performance; cross-validation & metrics
19. Deploy Model — deploy to production; create APIs / services
20. Monitor & Improve — monitor performance; retrain & improve models

This process iterates — refining data, features, models, and strategies continuously.

2. Core Languages

- Python — most popular for Data Science; libraries: NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, TensorFlow, PyTorch; great for analytics, ML, DL, NLP
- R — strong for statistics & analysis; libraries: dplyr, ggplot2, caret, tidyr, Shiny; excellent for data visualization & statistical modeling
- SQL — essential for data extraction, transformation & aggregation; used to query databases, data warehouses; foundation for data wrangling
- Other useful languages (as needed): Julia, Scala, Java, C++

3. Deployment

Path: Trained Model → Package the Model → Deploy → Users/Applications, with Monitor & Update as an ongoing loop.

Deployment Options

- Web APIs — expose model via REST API (FastAPI, Flask); integrate with web & mobile apps
- Cloud Platforms — AWS SageMaker, Google Vertex AI, Azure ML; managed infrastructure & auto-scaling
- Containers — Dockerize the model; consistent environment everywhere
- Batch / Scheduled — run predictions on schedules; useful for reports, scoring, ETL pipelines
- Edge / On-device — deploy on mobile, IoT, or edge devices; for real-time inference

Tools Ecosystem (Examples)

- Data Collection: APIs, Web Scraping, Databases
- Data Processing: Pandas, NumPy, Spark
- Machine Learning: Scikit-learn, XGBoost, LightGBM
- Deep Learning: TensorFlow, PyTorch, Keras
- Visualization: Matplotlib, Seaborn, Plotly, Tableau
- Deployment & MLOps: MLflow, Kubeflow, Airflow, Docker

The Goal: Use the right framework, the right languages, and the right deployment strategy to turn data into intelligent, real-world solutions.

Data Ingestion Techniques: Batch vs Streaming

29 June 2026

Data ingestion is the process of gathering, managing, and utilizing data efficiently. It plays a foundational role in the data processing pipeline.

- Batch Processing — processes a large amount of data at scheduled intervals.
- Real-time Processing — processes data immediately as it arrives.
- Micro-batching — processes small batches of data at frequent intervals, providing near real-time results.

Batch Processing

Definition: Batch processing collects data over a period of time and processes it all at once.

Examples: daily sales report, monthly bank statement, payroll processing.

Advantages: efficient for large volumes of data; cost-effective; simple to manage.

Disadvantages: results are delayed; not suitable for applications requiring instant updates.

Real-Time Processing (Stream Processing)

Definition: real-time processing processes data immediately as it arrives, with little or no delay.

Examples: UPI payment notifications, credit card fraud detection, live GPS tracking, stock market updates.

Advantages: instant results; supports live monitoring and decision-making.

Disadvantages: more complex to implement; higher infrastructure cost.

Micro-Batching

Definition: micro-batching processes data in small batches at short, regular intervals (e.g. every 5, 10, or 30 seconds). It provides near real-time processing.

Examples: website click analysis every 10 seconds; IoT sensor data processing every 30 seconds; log analysis every minute.

Advantages: faster than batch processing; easier to implement than true real-time processing.

Disadvantages: not true real-time; a small processing delay still exists.

Feature	Batch Processing	Real-Time Processing	Micro-Batching
Processing Method	Large batches	One record at a time	Small batches
Speed	Slow	Very Fast	Fast
Delay	Minutes/Hours	Milliseconds/Seconds	Few Seconds
Cost	Low	High	Medium
Best For	Scheduled reports	Live applications	Near real-time analytics

Source: chatgpt.com conversation

The Data Ingestion Workflow

Data Sources

- > Extract / Collect Data
- > Staging Area
- > Data Validation
- > Data Transformation
- > Load
- > Data Warehouse / Database / Data Lake
- > Analysis / Reporting / Machine Learning

Staging is a temporary storage area where raw data is stored before validation, cleaning, transformation, and loading into the final destination.

Validate

Check for: missing values, duplicate records, incorrect data types, invalid records.

Transform

Prepare the data by: cleaning, filtering, renaming columns, converting data types, aggregating data.

Load

Load the processed data into: database, data warehouse, or data lake.

Analysis

The stored data is used for: Business Intelligence (BI), dashboards, machine learning, data analytics.

Data Sources -> Data Ingestion -> Staging -> Data Validation -> Data Transformation
-> Data Loading -> Data Storage -> Data Monitoring
-> Data Analysis / Reporting / Machine Learning

Data Monitoring

Data monitoring means continuously checking the health and quality of the data.

- Are new records arriving on time?
- Are there missing values?

- Are there duplicate records?
- Did the ETL pipeline fail?
- Is the data accurate and complete?

Data Lake vs Data Warehouse

Data Lake

Definition: a Data Lake is a centralized repository that stores raw data in its original format until it is needed.

Stores:

- Structured data (CSV, SQL tables)
- Semi-structured data (JSON, XML)
- Unstructured data (images, videos, audio, PDFs)

CSV, JSON, Images, Videos, Logs

->

Data Lake

Best for: Machine Learning, Big Data Analytics, AI Projects.

Data Warehouse

Definition: a Data Warehouse is a centralized repository that stores cleaned, transformed, and structured data for reporting and business intelligence.

Stores: only processed and structured data.

Raw Data

-> ETL (Clean & Transform)

-> Data Warehouse

-> Power BI / Reports / SQL Queries

Best for: business reports, dashboards, SQL queries, decision making.

Common Batch Processing Tools

Tool	Purpose
Apache Spark	Large-scale batch data processing
Apache Hadoop (MapReduce)	Distributed batch processing of big data
Apache Hive	SQL-based batch processing on Hadoop
Apache Airflow	Schedules and automates batch workflows (orchestration)
Apache Oozie	Workflow scheduler for Hadoop batch jobs
Talend	ETL and batch data integration
Informatica PowerCenter	Enterprise ETL and batch processing
AWS Glue	Cloud ETL and batch processing
Azure Data Factory	Cloud data integration and batch pipelines
Google Cloud Dataflow	Supports both batch and stream processing

| Source: chatgpt.com conversation

Most common tools: Apache Spark (batch processing, ETL, analytics), Hadoop MapReduce (large-scale batch processing), Apache Airflow (scheduling & automating batch jobs), AWS Glue (cloud-based batch ETL), Azure Data Factory (batch data pipelines).

Stream Processing (Real-Time Processing)

Stream Processing is the continuous processing of data as it is generated. Instead of waiting to collect all the data, each event or record is processed immediately or with very low latency.

Examples

- UPI payment processing
- Credit card fraud detection
- Live stock market updates
- GPS navigation
- Social media feeds
- IoT sensor monitoring

Advantages

- Provides real-time insights.
- Detects issues immediately.
- Supports instant decision-making.
- Low latency (milliseconds to seconds).

Disadvantages

- More complex to build and maintain.
- Higher infrastructure cost.
- Requires continuous monitoring.

Common Stream Processing Tools

Tool	Purpose
Apache Kafka	Distributed event streaming platform
Apache Spark Streaming	Real-time stream processing
Apache Flink	High-performance stream processing
Apache Storm	Real-time data processing
Apache Pulsar	Event streaming and messaging
Amazon Kinesis	AWS real-time data streaming
Azure Event Hubs	Microsoft Azure event streaming
Google Cloud Pub/Sub	Real-time messaging and event ingestion

| Source: chatgpt.com conversation

Apache Kafka

Apache Kafka is an open-source distributed event streaming platform used to collect, store, and process real-time data streams.

In simple terms: Kafka acts like a high-speed messaging system that transfers data from one application to another in real time.

Why is Kafka Used?

- Real-time data streaming
- High-speed data transfer
- Handling millions of events/messages
- Building data pipelines
- Connecting different applications

How Kafka Works

Producer ---> Kafka Broker ---> Consumer
(Sends) (Stores) (Reads)

Kafka Architecture: Producer → Topic → Broker → Consumer

- Producer: sends messages to Kafka.
- Topic: category where messages are stored.
- Broker: Kafka server that stores and manages messages.
- Consumer: reads messages from Kafka.

Main Components

Component	Description
Producer	Sends data to Kafka.
Consumer	Reads data from Kafka.
Broker	Kafka server that stores messages.
Topic	Logical channel for messages.
Partition	Divides a topic for parallel processing.
Offset	Unique position of each message in a partition.
Consumer Group	Multiple consumers sharing the workload.

Source: chatgpt.com conversation

Offset

- A unique sequential number assigned to each message in a partition.
- Tracks the position of messages.
- Helps consumers resume reading from the last processed message.

Partition

- Splits a topic into smaller parts.
- Enables parallel processing.
- Improves scalability and performance.

Consumer Group

- A group of consumers reading the same topic.
- Each partition is processed by only one consumer in the group.
- Increases processing speed and fault tolerance.

One-line Definitions

- Producer: sends messages to Kafka.
- Consumer: reads messages from Kafka.
- Broker: Kafka server that stores messages.
- Topic: category/channel where messages are stored.
- Partition: division of a topic for parallel processing.
- Offset: unique position number of a message in a partition.
- Consumer Group: multiple consumers working together to process messages.

Example: E-commerce Website

Scenario: a customer places an order on an e-commerce website.

Order Details: Order ID: 101 | Customer: Rahul | Product: iPhone 16 | Amount: ₹80,000

Step 1: Producer

The e-commerce website acts as the Producer and sends the order details to Kafka.

Step 2: Topic

The Producer sends the order to the "Orders" topic.

Step 3: Broker

The Kafka Broker receives and stores the message.

Step 4: Partition & Offset

Orders Topic – Partition 0

Offset 0 -> Order #101

Offset 1 -> Order #102

Offset 2 -> Order #103

Offset 3 -> Order #104

Offset: unique position number of each message in a partition.

Step 5: Consumers

Multiple services read the same order independently: Inventory, Payment, Shipping, and Analytics consumers.

- Inventory Consumer — reduces product stock.
- Payment Consumer — verifies payment, generates invoice.
- Shipping Consumer — creates shipment, assigns courier.
- Analytics Consumer — updates sales dashboard, calculates revenue.

Example: Courier Booking Workflow

21. Customer books a courier.

22. The Courier App (Producer) sends the booking event to Kafka.
23. Kafka stores the event in the Courier_Orders topic.
24. The topic is divided into partitions.
25. Each message gets an offset.

Different consumer groups read the same event:

- Pickup Service → assigns pickup agent.
- Tracking Service → updates package location.
- Delivery Service → assigns delivery executive.
- Notification Service → sends SMS/email updates.

Components

- Producer → Courier Website/App
- Broker → Apache Kafka Server
- Topic → Courier_Orders
- Partition → divides the topic for parallel processing
- Offset → unique position of each message
- Consumer Groups → Pickup, Tracking, Delivery, Notification services

ZooKeeper (Apache ZooKeeper)

ZooKeeper is a centralized service that manages and coordinates Kafka brokers in a Kafka cluster. It helps Kafka keep track of brokers, topics, partitions, and leader election.

Example: Banking App Fraud Alerts

Step 1 — Customer makes a transaction, e.g. transfers ₹50,000. The Banking App acts as the Producer.

Step 2 — Producer sends the transaction details to the Kafka topic "Transactions".

Step 3 — Kafka stores the message:

```
Transactions Topic
Offset 0 -> ₹5,000
Offset 1 -> ₹50,000
Offset 2 -> ₹1,20,000
```

Step 4 — Multiple consumers process the same transaction:

- Fraud Detection Consumer — checks if the transaction is suspicious, detects unusual login location or large amount, flags possible fraud.
- Account Service Consumer — updates the customer's account balance.
- Notification Service Consumer — sends an SMS, email, or push notification, e.g. "₹50,000 debited from your account. If this wasn't you, contact the bank immediately."

ETL vs ELT: Building Robust Data Pipelines Using Apache Airflow

30 June 2026

Apache Airflow is an open-source workflow orchestration tool used to automate, schedule, and monitor data pipelines.

Why Use Airflow?

- Automates ETL and ELT workflows
- Schedules pipelines (hourly, daily, weekly)
- Monitors task execution
- Handles task dependencies
- Supports retries on failure
- Easy to scale and maintain

Airflow Components

- DAG (Directed Acyclic Graph) — defines the workflow and task dependencies.
- Task — a single unit of work (e.g. extract data).
- Operator — specifies what a task does (Python, SQL, Bash, etc.).
- Scheduler — triggers workflows based on schedules.
- Executor — runs the tasks.
- Web UI — monitors pipeline status and logs.

Apache Airflow is a workflow orchestration (pipeline management) tool used to schedule, automate, and monitor data pipelines. It is used in batching, not in real-time.

What Airflow Does

- Schedules pipelines (e.g. every day at 2 AM)
- Executes tasks in the correct order
- Manages task dependencies
- Retries failed tasks automatically
- Monitors pipeline status through a web interface

Airflow Core Concepts

- DAG: Directed Acyclic Graph
- Task: one step in the pipeline
- Operator: type of task (Python, Bash, SQL, etc.)

1. Scheduler

The Scheduler is the brain of Apache Airflow. It checks the DAGs, determines when tasks should run, and triggers them according to the schedule and dependencies.

Functions: monitors DAGs; schedules tasks; triggers task execution; ensures task dependencies are met.

Example: if a DAG is scheduled to run every day at 2:00 AM, the Scheduler starts it at that time.

2. Metadata Database

The Metadata Database stores all the information Airflow needs to manage workflows.

It stores: DAG information; task status (success, failed, running); execution history; logs and scheduling information.

Common databases used: SQLite (development); PostgreSQL (production); MySQL (production).

3. Web UI

The Web UI is Airflow's web-based interface used to monitor and manage workflows.

Functions: view DAGs; monitor task status; trigger DAGs manually; view logs; retry failed tasks.

Creating an Airflow Project — Steps

```
dags/  
include/  
plugins/  
tests/  
airflow_settings.yaml  
Dockerfile  
packages.txt  
requirements.txt
```

Project — Bike Sharing Dataset

| 02 July 2026

1. What is Your Data Source?

The dataset is obtained from the UCI Machine Learning Repository. It contains hourly records of bike-sharing rentals collected during 2011 and 2012.

2. Why Did You Choose It?

I chose this dataset because it is a real-world dataset with a sufficient number of records (17,379 rows) and multiple features. It is ideal for learning and implementing an ETL pipeline, including data extraction, cleaning, transformation, and loading into a database.

3. What Information Does It Contain?

The dataset contains hourly bike rental information along with weather and calendar-related features. It includes:

- Record ID
- Date
- Season
- Year
- Month
- Hour
- Holiday
- Day of the week
- Working day

- Weather situation
- Temperature
- Feels-like temperature
- Humidity
- Wind speed
- Number of casual users
- Number of registered users
- Total bike rentals

4. Is It Structured, Semi-Structured, or Unstructured?

The dataset is structured data because it is organized into rows and columns with predefined data types.

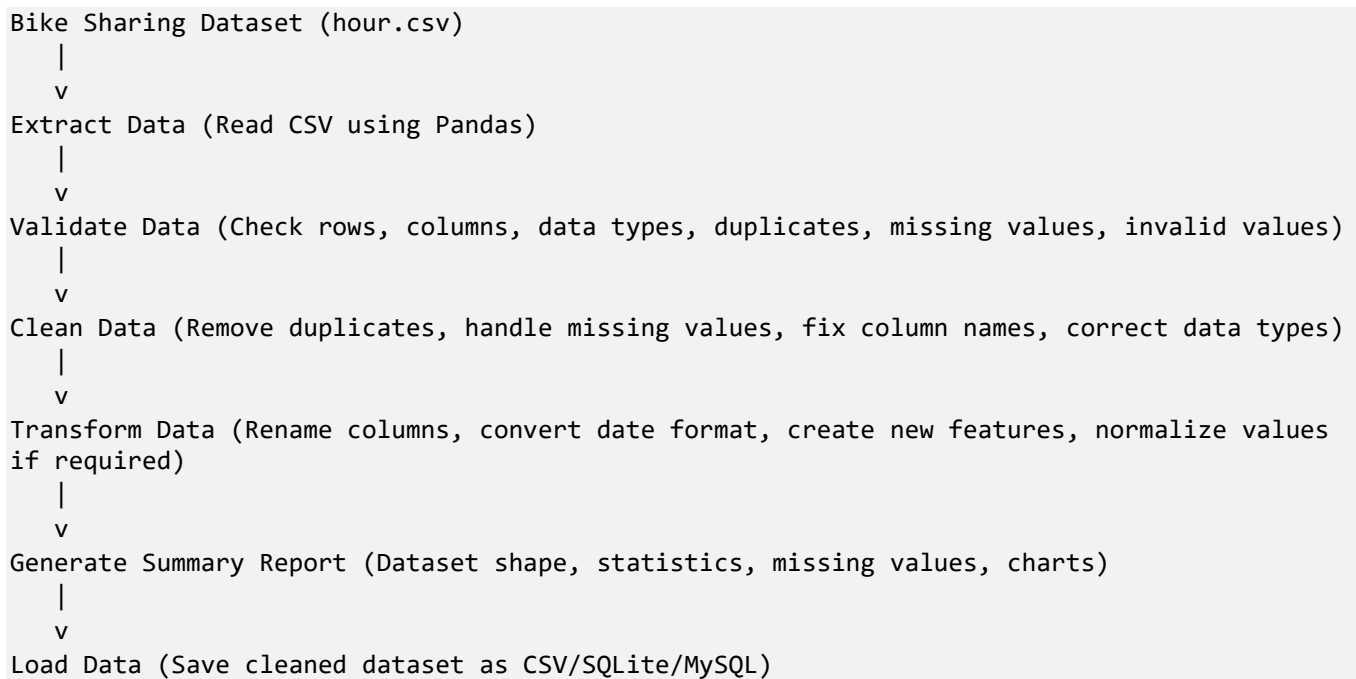
5. What Format Is the Data?

The data is stored in CSV (Comma-Separated Values) format.

6. How Frequently Is the Data Updated?

This is a historical dataset and is not updated regularly. It contains hourly bike-sharing data collected during 2011 and 2012.

Flow Diagram



Feature Engineering and Visualization

03 July 2026

Why Is It Crucial for Building Successful Machine Learning Models?

- Improves Model Accuracy — reveals hidden patterns in the data and enables more accurate predictions
- Reduces Overfitting — by selecting and transforming features that generalize well across different datasets
- Simplifies the Model — reduces complexity
- Enhances Interpretability

Real-World Examples

- E-commerce systems
- Credit scoring
- Healthcare predictive models

Data Types: Continuous and Discrete

Continuous Features — features that take any value within a range, often representing measurements or quantities (e.g. height, weight, temperature).

Concept	Meaning	Example (Bike Sharing dataset)
Continuous Data	Numeric values that can take any value within a range (including decimals).	Temperature = 0.24, Humidity = 0.64, Wind_Speed = 0.1940
Discrete Data	Countable values (integers or categories).	Hour = 10, Total_Rentals = 172, Season = Spring
Normalization	Scaling numeric values to a common range (usually 0–1).	Temperature = 0.24, Humidity = 0.64

Source: chatgpt.com conversation

Transformations for Continuous Data

Transformation	Purpose	Example
Normalization (Min-Max Scaling)	Scale values to 0–1	Temperature: 20°C → 0.50
Standardization (Z-score Scaling)	Mean = 0, standard deviation = 1	Salary → z-score
Log Transformation	Reduce skewness in highly skewed data	Income, Sales

Visualization for Continuous Data

- Histogram
- Density plot
- Tools: Matplotlib, Seaborn

Techniques for Handling Discrete Data

- One-Hot Encoding
- Label Encoding

One-Hot Encoding

One-Hot Encoding is a technique used to convert categorical (text) data into multiple binary (0 or 1) columns. Each unique category gets its own column — a new column is created.

Original Color column: Red, Blue, Green, Red

After One-Hot Encoding:

Color_Blue	Color_Green	Color_Red
0	0	1
1	0	0
0	1	0
0	0	1

Source: chatgpt.com conversation

Label Encoding

Label Encoding is a preprocessing technique used in machine learning to convert categorical (text) data into numerical values by assigning each unique category an integer — the column is replaced with numerical values.

Color	Label Encoded
Red	0
Blue	1
Green	2
Red	0
Green	2

Source: chatgpt.com conversation

Feature Extraction

Feature Extraction is the process of creating or extracting new, meaningful features from raw data while reducing unnecessary information. It helps machine learning models learn patterns more effectively.

1. Principal Component Analysis (PCA)

- Reduces the number of features while retaining most of the important information (variance).
- Used for high-dimensional numerical data.

Example: 100 features → PCA → 10 principal components.

Use case: image recognition, data visualization, dimensionality reduction.

2. Linear Discriminant Analysis (LDA)

- Reduces dimensions while maximizing the separation between classes.
- Used for supervised classification problems.

Use case: face recognition, medical diagnosis.

3. Text Vectorization

Converts text into numerical features. Techniques:

- Bag of Words (BoW) — counts the occurrence of each word.
- TF-IDF (Term Frequency–Inverse Document Frequency) — gives more importance to unique words.
- Word Embeddings — represents words as dense vectors (e.g. Word2Vec, GloVe, FastText).

Example: "I love AI" → [0.2, 0.8, 0.5, ...]

4. Image Feature Extraction

Extracts meaningful information from images. Techniques:

- Edge detection
- Color histograms
- Texture features
- CNN (Convolutional Neural Networks)

Use case: face detection, object recognition.

5. Audio Feature Extraction

Extracts useful information from audio signals. Examples:

- MFCC (Mel-Frequency Cepstral Coefficients)
- Spectrogram
- Chroma features

Use case: speech recognition, music classification.

6. Autoencoders

- A type of neural network that learns a compressed representation of data.
- Commonly used for dimensionality reduction and anomaly detection.

7. Independent Component Analysis (ICA)

- Separates mixed signals into independent components.

Use case: noise removal, signal processing.

Binning

Binning (also called discretization) is a feature engineering technique that groups continuous numerical values into a small number of intervals (bins), instead of using exact values.

Example — Age column:

Age	Age Group
18	Young
22	Young
35	Adult
47	Middle-aged
60	Senior

Feature Selection

Feature Selection is the process of selecting the most relevant features (columns) from a dataset and removing the irrelevant or redundant ones. Unlike feature extraction, it does not create new features — it only chooses the best existing features.

Example — for predicting salary, you might keep Age and Experience, and remove Name and Employee ID because they may not help the model.

Wrapper Method (Feature Selection)

A Wrapper Method is a feature selection technique that selects the best subset of features by repeatedly training and evaluating a machine learning model. Instead of using only statistical tests (like filter methods), it "wraps" around the machine learning algorithm to find the best combination of features.

How It Works

26. Select a subset of features.
27. Train the model using those features.
28. Evaluate the model's performance.
29. Add or remove features.
30. Repeat until the best feature subset is found.

Types of Wrapper Methods

1. Forward Selection

- Starts with no features.
- Adds one feature at a time.
- Keeps the feature that improves model performance the most.
- Stops when adding more features no longer improves the model.

Example: Features: Age, Salary, Experience, City

```
Start -> {}  
Add Age -> {Age}  
Add Experience -> {Age, Experience}  
Stop
```

2. Backward Elimination

- Starts with all features.
- Removes one feature at a time.
- Removes the least useful feature.
- Stops when only important features remain.

```
{Age, Salary, Experience, City}  
Remove City -> {Age, Salary, Experience}  
Remove Salary -> {Age, Experience}
```

3. Recursive Feature Elimination (RFE)

- Starts with all features.
- Trains the model.
- Ranks features by importance.
- Removes the least important feature.
- Repeats until the desired number of features remains.

Advantages

- Usually provides high accuracy because it evaluates actual model performance.
- Finds the best feature combination for a specific model.

Disadvantages

- Computationally expensive because the model is trained many times.
- Slow for datasets with many features.

Iris Dataset

The Iris dataset is one of the most famous datasets in machine learning. It was introduced by the British statistician Ronald A. Fisher in 1936 and is commonly used for classification tasks.

The Iris dataset is a benchmark machine learning dataset containing 150 iris flower samples with four numerical features and three species. It is widely used for learning classification algorithms and data visualization techniques.